

REFERENCE

Interactive mode

Complete reference for keyboard shortcuts, input modes, and interactive features in Claude Code sessions.

Keyboard shortcuts

ⓘ Keyboard shortcuts may vary by platform and terminal. In [fullscreen rendering](#), press `?` in the transcript viewer to see available shortcuts there.

macOS users: Option/Alt key shortcuts (`Alt+B` , `Alt+F` , `Alt+Y` , `Alt+M` , `Alt+P`) require configuring Option as Meta in your terminal:

- **iTerm2:** Settings → Profiles → Keys → General → set Left/Right Option key to “Esc+”
- **Apple Terminal:** Settings → Profiles → Keyboard → check “Use Option as Meta Key”
- **VS Code:** set `"terminal.integrated.macOptionIsMeta": true` in VS Code settings

See [Terminal configuration](#) for details.

General controls

Shortcut	Description	Context
<code>Ctrl+C</code>	Interrupt, or clear input	Interrupts a running operation. If nothing is running, the first press clears the prompt input and a second press exits Claude Code
<code>Ctrl+X</code> <code>Ctrl+K</code>	Stop all running <u>background subagents</u> in this session. Press twice within 3 seconds to confirm	Subagent control
<code>Ctrl+D</code>	Exit Claude Code session	EOF signal
<code>Ctrl+G</code> or <code>Ctrl+X</code> <code>Ctrl+E</code>	Open in default text editor	Edit your prompt or custom response in your default text editor. <code>Ctrl+X</code> <code>Ctrl+E</code> is the readline-native binding. Turn on Show last response in external editor in <code>/config</code> to prepend Claude’s previous reply as <code>#</code> -commented context above your prompt; the comment block is stripped when you save

Shortcut	Description	Context
<code>Ctrl+L</code>	Redraw screen	Forces a full terminal redraw. Input and conversation history are kept. Use this to recover if the display becomes garbled or partially blank
<code>Ctrl+O</code>	Toggle transcript viewer	Shows detailed tool usage and execution. Also expands MCP calls, which collapse to a single line like “Called slack 3 times” by default
<code>Ctrl+R</code>	Reverse search command history	Search through previous commands interactively
<code>Ctrl+V</code> or <code>Cmd+V</code> (iTerm2) or <code>Alt+V</code> (Windows and WSL)	Paste image from clipboard	Inserts an <code>[Image #N]</code> chip at the cursor so you can reference it positionally in your prompt. On WSL, both <code>Ctrl+V</code> and <code>Alt+V</code> are bound; use <code>Alt+V</code> if your terminal intercepts <code>Ctrl+V</code>
<code>Ctrl+B</code>	Background running tasks	Backgrounds Bash commands and agents. Tmux users press twice
<code>Ctrl+T</code>	Toggle task list	Show or hide the <u>task list</u> in the terminal status area
<code>Left/Right arrows</code>	Cycle through dialog tabs	Navigate between tabs in permission dialogs and menus
<code>Up/Down arrows</code> or <code>Ctrl+P</code> / <code>Ctrl+N</code>	Move cursor or navigate command history	When the input spans more than one visual row, whether wrapped or multiline, first moves the cursor within the prompt. Once the cursor is on the first or last visual row, pressing again navigates command history. As of v2.1.169, wrapped single-line input behaves the same as multiline
<code>Esc</code>	Interrupt Claude	Stop the current response or tool call mid-turn so you can redirect. Claude keeps the work done so far
<code>Esc</code> + <code>Esc</code>	Clear input draft, or rewind	When the prompt input contains text, double <code>Esc</code> clears it and saves the draft to history so <code>Up</code> recalls it. When the input is empty, double <code>Esc</code> opens the <u>rewind menu</u> to restore or summarize code and conversation from a previous point
<code>Shift+Tab</code> or <code>Alt+M</code> (some configurations)	Cycle permission modes	Cycle through <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , and any modes you have enabled, such as <code>auto</code> or <code>bypassPermissions</code> . See <u>permission modes</u> .
<code>Option+P</code> (macOS) or <code>Alt+P</code> (Windows/Linux)	Switch model	Switch models without clearing your prompt
<code>Option+T</code> (macOS) or <code>Alt+T</code> (Windows/Linux)	Toggle extended thinking	Enable or disable extended thinking mode. Has no effect on Fable 5, which always uses extended thinking. As of v2.1.132 this shortcut works on macOS without configuring Option as Meta
<code>Option+O</code> (macOS) or <code>Alt+O</code> (Windows/Linux)	Toggle fast mode	Enable or disable <u>fast mode</u>

Text editing

Shortcut	Description	Context
Ctrl+A	Move cursor to start of current line	In multiline input, moves to the start of the current logical line
Ctrl+E	Move cursor to end of current line	In multiline input, moves to the end of the current logical line
Ctrl+K	Delete to end of line	Stores deleted text for pasting
Ctrl+U	Delete from cursor to line start	Stores deleted text for pasting. Repeat to clear across lines in multiline input. On macOS, terminal emulators including iTerm2 and Terminal.app map Cmd+Backspace to this shortcut
Ctrl+W	Delete previous word	Stores deleted text for pasting. On Windows, Ctrl+Backspace also deletes the previous word
Ctrl+Y	Paste deleted text	Paste text deleted with Ctrl+K , Ctrl+U , or Ctrl+W
Alt+Y (after Ctrl+Y)	Cycle paste history	After pasting, cycle through previously deleted text. Requires <u>Option as Meta</u> on macOS
Alt+B	Move cursor back one word	Word navigation. Requires <u>Option as Meta</u> on macOS
Alt+F	Move cursor forward one word	Word navigation. Requires <u>Option as Meta</u> on macOS

Theme and display

Shortcut	Description	Context
Ctrl+T	Toggle syntax highlighting for code blocks	Only works inside the /theme picker menu. Controls whether code in Claude’s responses uses syntax coloring

Multiline input

Method	Shortcut	Context
Quick escape	\ + Enter	Works in all terminals
Option key	Option+Enter	After enabling <u>Option as Meta</u> on macOS

Method	Shortcut	Context
Shift+Enter	Shift+Enter	Native in iTerm2, WezTerm, Ghostty, Kitty, Warp, Apple Terminal, Windows Terminal
Control sequence	Ctrl+J	Works in any terminal without configuration
Paste mode	Paste directly	For code blocks, logs

💡 Shift+Enter works without configuration in iTerm2, WezTerm, Ghostty, Kitty, Warp, Apple Terminal, and Windows Terminal. For VS Code, Cursor, Devin Desktop, Alacritty, and Zed, run `/terminal-setup` to install the binding.

Quick commands

Shortcut	Description	Notes
/ at start	Command or skill	See <u>commands</u> and <u>skills</u>
! at start	Shell mode	Run a command directly, add its output to the session, and have Claude respond to it
@	File path mention	Trigger file path autocomplete

Transcript viewer

When the transcript viewer is open (toggled with `Ctrl+O`), these shortcuts are available. In fullscreen rendering, press `?` to show the full shortcut reference panel inside the viewer. `Ctrl+E` can be rebound via `transcript:toggleShowAll`.

Shortcut	Description
?	Toggle the keyboard shortcut help panel. Requires <u>fullscreen rendering</u>
{ / }	Jump to the previous or next user prompt, like vim paragraph motion. Requires <u>fullscreen rendering</u>
Ctrl+E	Toggle show all content
[	Write the full conversation to your terminal's native scrollbar so <code>Cmd+F</code> , tmux copy mode, and other native tools can search it. Requires <u>fullscreen rendering</u>
v	Write the conversation to a temporary file and open it in <code>\$VISUAL</code> or <code>\$EDITOR</code> . Requires <u>fullscreen rendering</u>

Shortcut	Description
<code>q</code> , <code>Ctrl+C</code> , <code>Esc</code>	Exit transcript view. All three can be rebound via <code>transcript:exit</code>

Voice input

Shortcut	Description	Notes
Hold or tap <code>Space</code>	Voice dictation	Requires voice dictation to be enabled. Hold to record, or run <code>/voice tap</code> for tap-to-toggle. Rebindable

Commands

Type `/` in Claude Code to see all available commands, or type `/` followed by any letters to filter. The `/` menu shows everything you can invoke: built-in commands, bundled and user-authored **skills**, and commands contributed by **plugins** and **MCP servers**. Not all built-in commands are visible to every user since some depend on your platform or plan.

See the **commands reference** for the full list of commands included in Claude Code.

Vim editor mode

Enable vim-style editing via `/config` → Editor mode.

Mode switching

Command	Action	From mode
<code>Esc</code>	Enter NORMAL mode	INSERT, VISUAL
<code>i</code>	Insert before cursor	NORMAL
<code>I</code>	Insert at beginning of line	NORMAL
<code>a</code>	Insert after cursor	NORMAL
<code>A</code>	Insert at end of line	NORMAL
<code>o</code>	Open line below	NORMAL
<code>O</code>	Open line above	NORMAL

Command	Action	From mode
<code>v</code>	Start character-wise visual selection	NORMAL
<code>V</code>	Start line-wise visual selection	NORMAL

Navigation (NORMAL mode)

Command	Action
<code>h / j / k / l</code>	Move left/down/up/right
<code>Space</code>	Move right
<code>w</code>	Next word
<code>e</code>	End of word
<code>b</code>	Previous word
<code>0</code>	Beginning of line
<code>\$</code>	End of line
<code>^</code>	First non-blank character
<code>gg</code>	Beginning of input
<code>G</code>	End of input
<code>f{char}</code>	Jump to next occurrence of character
<code>F{char}</code>	Jump to previous occurrence of character
<code>t{char}</code>	Jump to just before next occurrence of character
<code>T{char}</code>	Jump to just after previous occurrence of character
<code>;</code>	Repeat last f/F/t/T motion
<code>,</code>	Repeat last f/F/t/T motion in reverse
<code>/</code>	Open reverse history search, same as <code>Ctrl+R</code> . As of v2.1.191, the empty search prompt shows a hint: press <code>Esc</code> then <code>i</code> then <code>/</code> to open the command menu instead

❗ In vim normal mode, if the cursor is at the beginning or end of input and can't move further, `j / k` and the arrow keys navigate command history instead.

Editing (NORMAL mode)

Command	Action
x	Delete character
dd	Delete line
D	Delete to end of line
dw / de / db	Delete word/to end/back
cc	Change line
C	Change to end of line
cw / ce / cb	Change word/to end/back
yy / Y	Yank (copy) line
yw / ye / yb	Yank word/to end/back
p	Paste after cursor
P	Paste before cursor
>>	Indent line
<<	Dedent line
J	Join lines
u	Undo
.	Repeat last change

Text objects (NORMAL mode)

Text objects work with operators like `d` , `c` , and `y` :

Command	Action
iw / aw	Inner/around word
iW / aW	Inner/around WORD (whitespace-delimited)
i" / a"	Inner/around double quotes
i' / a'	Inner/around single quotes
i(/ a(	Inner/around parentheses

Command	Action
i[/ a[	Inner/around brackets
i{ / a{	Inner/around braces

Visual mode

Press `v` for character-wise selection or `V` for line-wise selection. Motions extend the selection, and operators act on it directly.

Command	Action
d / x	Delete selection
y	Yank selection
c / s	Change selection
p	Replace selection with register contents
r{char}	Replace every selected character with <code>{char}</code>
~ / u / U	Toggle, lowercase, or uppercase selection
> / <	Indent or dedent selected lines
J	Join selected lines
o	Swap cursor and anchor
iw / aw / i" / ...	Select a text object
v / V	Toggle between character-wise and line-wise, or exit

Block-wise visual mode with `Ctrl+V` is not supported.

Command history

Claude Code maintains command history for the current session:

- Input history is stored per working directory
- Input history resets when you run `/clear` to start a new session. The previous session's conversation is preserved and can be resumed.
- Submitting the same prompt twice in a row records one history entry, so pressing Up steps to

the previous distinct prompt

- Use Up/Down arrows to navigate (see keyboard shortcuts above)
- History expansion with `!` is disabled by default

Reverse search with Ctrl+R

Press `Ctrl+R` to interactively search through your command history:

1. **Start search:** press `Ctrl+R` to activate reverse history search
2. **Type query:** enter text to search for in previous commands. The search term is highlighted in matching results
3. **Navigate matches:** press `Ctrl+R` again to cycle through older matches
4. **Change scope:** search defaults to prompts from all projects. Press `Ctrl+S` to cycle the scope through this session, this project, and all projects
5. **Accept match:**
 - Press `Tab` or `Esc` to accept the current match and continue editing
 - Press `Enter` to accept and execute the command immediately
6. **Cancel search:**
 - Press `Ctrl+C` to cancel and restore your original input
 - Press `Backspace` on empty search to cancel

The search loads the 100 most recent unique prompts in the selected scope, with duplicates collapsed to the newest occurrence. Matching prompts display with the search term highlighted, so you can find and reuse previous inputs.

Background Bash commands

Claude Code supports running Bash commands in the background, allowing you to continue working while long-running processes execute.

How backgrounding works

When Claude Code runs a command in the background, it runs the command asynchronously and immediately returns a background task ID. Claude Code can respond to new prompts while the command continues executing in the background.

To run commands in the background, you can either:

- Prompt Claude Code to run a command in the background
- Press `Ctrl+B` to move a regular Bash tool invocation to the background. Tmux users must press `Ctrl+B` twice due to tmux's prefix key.

Key features:

- Output is written to a file and Claude can retrieve it using the Read tool
- Background tasks have unique IDs for tracking and output retrieval
- Background tasks are automatically cleaned up when Claude Code exits
- Background tasks are automatically terminated if output exceeds 5GB, with a note in stderr explaining why

To disable all background task functionality, set the `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` environment variable to `1`. See [Environment variables](#) for details.

Common backgrounded commands:

- Build tools (webpack, vite, make)
- Package managers (npm, yarn, pnpm)
- Test runners (jest, pytest)
- Development servers
- Long-running processes (docker, terraform)

Shell mode with `!` prefix

Run shell commands directly without going through Claude by prefixing your input with `!`:

```
! npm test
! git status
! ls -la
```

Shell mode:

- Adds the command and its output to the conversation context
- Shows real-time progress and output

- Supports the same `Ctrl+B` backgrounding for long-running commands
- Doesn't require Claude to interpret or approve the command
- Supports history-based autocomplete: type a partial command and press `Tab` to complete from previous `!` commands in the current project
- Exit with `Escape`, `Backspace`, or `Ctrl+U` on an empty prompt
- Pasting text that starts with `!` into an empty prompt enters shell mode automatically, matching typed `!` behavior

As of v2.1.186, Claude responds to the command output automatically once it lands in the transcript, so you can run `! npm test` and get an explanation of the failures without a second prompt. The response costs the same as sending a normal prompt. To restore the earlier behavior where the output is added to context without a response, set `respondToBashCommands` to `false` in `settings.json`. Before v2.1.186, shell mode always added output to context without a response.

This is useful for quick shell operations while maintaining conversation context.

Prompt suggestions

When you first open a session, a grayed-out example command appears in the prompt input to help you get started. Claude Code picks this from your project's git history, so it reflects files you've been working on recently.

After Claude responds, suggestions continue to appear based on your conversation history, such as a follow-up step from a multi-part request or a natural continuation of your workflow.

- Press `Tab` or `Right arrow` to place the suggestion in the prompt input, then `Enter` to submit
- Start typing to dismiss it

The suggestion runs as a background request that reuses the parent conversation's prompt cache, so the additional cost is minimal. Claude Code skips suggestion generation when the cache is cold to avoid unnecessary cost.

Suggestions are automatically skipped after the first turn of a conversation and in plan mode.

In print mode they are off by default. Pass `--prompt-suggestions` with `--output-format stream-json --verbose` to emit a `prompt_suggestion` message after each turn instead.

To disable prompt suggestions entirely, set the environment variable or toggle the setting in `/config`:

```
export CLAUDE_CODE_ENABLE_PROMPT_SUGGESTION=false
```

Side questions with `/btw`

Use `/btw` to ask a quick question about your current work without adding to the conversation history. This is useful when you want a fast answer but don't want to clutter the main context or derail Claude from a long-running task.

```
/btw what was the name of that config file again?
```

Side questions have full visibility into the current conversation, so you can ask about code Claude has already read, decisions it made earlier, or anything else from the session. The question and answer are ephemeral: they appear in a dismissible overlay and never enter the conversation history.

- **Available while Claude is working:** you can run `/btw` even while Claude is processing a response. The side question runs independently and doesn't interrupt the main turn.
- **No tool access:** side questions answer only from what is already in context. Claude can't read files, run commands, or search when answering a side question.
- **Single response:** there are no follow-up turns in the overlay. To continue the thread, fork it into its own session with `f`.
- **Low cost:** the side question reuses the parent conversation's prompt cache, so the additional cost is minimal.

Earlier side questions from the same session appear as a dimmed list above the current answer. They stay out of the conversation history but remain visible in the overlay until you clear them.

Once the answer appears, the overlay accepts these keys.

Key	Action
Space , Enter , Escape	Dismiss the answer and return to the prompt
Up / Down	Scroll the answer
Left / Right	Step between this answer and your earlier <code>/btw</code> answers from the session. <code>Left</code> moves to older answers and <code>Right</code> returns toward the current one. Requires Claude Code v2.1.187 or later

Key	Action
<code>c</code>	Copy the answer to your clipboard as raw Markdown. Use this instead of mouse selection, which captures the hard-wrapped terminal rendering rather than the source text
<code>f</code>	Fork into a new session. The fork inherits the parent conversation plus this question and answer as real transcript turns, so you can continue with full tool access. The original session is preserved under <code>/resume</code> . Available in local sessions only
<code>x</code>	Clear the list of earlier <code>/btw</code> exchanges shown above the current answer

`/btw` is the inverse of a **subagent**: it sees your full conversation but has no tools, while a subagent has full tools but starts with an empty context. Use `/btw` to ask about what Claude already knows from this session; use a subagent to go find out something new.

Task list

When working on complex, multi-step work, Claude creates a task list to track progress. Tasks appear in the status area of your terminal with indicators showing what’s pending, in progress, or complete.

- Press `Ctrl+T` to toggle the task list view. The display shows up to 5 tasks at a time
- To see all tasks or clear them, ask Claude directly: “show me all tasks” or “clear all tasks”
- Tasks persist across context compactions, helping Claude stay organized on larger projects
- To share a task list across sessions, set `CLAUDE_CODE_TASK_LIST_ID` to use a named directory in `~/.claude/tasks/` : `CLAUDE_CODE_TASK_LIST_ID=my-project claude`

Session recap

When you return to the terminal after stepping away, Claude Code shows a one-line recap of what happened in the session so far. The recap generates in the background once at least three minutes have passed since the last completed turn and the terminal is unfocused, so it’s ready when you switch back. Recaps only appear once the session has at least three turns, and never twice in a row.

Run `/recap` to generate a summary on demand. To turn automatic recaps off, open `/config` and disable **Session recap**.

Session recap is on by default for every plan and provider. The recap is always skipped in non-interactive mode.

PR review status

When working on a branch with an open pull request, Claude Code displays a clickable PR link in the footer, such as “PR #446”. The link has a colored underline indicating the review state:

- Green: approved
- Yellow: pending review
- Red: changes requested
- Gray: draft

The badge disappears once the pull request merges or closes. `Cmd+click` (macOS) or `Ctrl+click` (Windows/Linux) the link to open the pull request in your browser. The status refreshes every 60 seconds, and immediately after a `gh pr` or `git push` command runs in the session.

❗ PR status requires the `gh` CLI to be installed and authenticated (`gh auth login`).

See also

- **Skills** - Custom prompts and workflows
- **Checkpointing** - Rewind Claude’s edits and restore previous states
- **CLI reference** - Command-line flags and options
- **Settings** - Configuration options
- **Memory management** - Managing CLAUDE.md files